

PEARLS AQI PREDICTOR

Final Technical Project Report End-to-End AQI Forecasting, Automation,
Explainability and Dashboard

Program	10P Shine Internship Program
Repository	github.com/hassantheshah/pearls-aqi-predictor
Primary city	Karachi, Pakistan
Forecast contract	Features at time $t \rightarrow$ AQI at time $t+1$
Forecast horizon	3 days (72 hourly steps) in the configured application
Verified against public repo	06 September 2026

Report standard. This report is written as a technical record, not a marketing summary. It distinguishes source-code implementation, recorded experiment results, and capabilities that were not independently verified as public production deployments. Regression metrics are reported as RMSE, MAE and R^2 ; they are not described as a percentage "accuracy."

Contents

- 1 Executive Summary
- 2 Project Objectives and Internship Deliverables
- 3 System Architecture
- 4 Data Sources and Preparation
- 5 Feature Engineering and Leakage Control
- 6 Forecasting and Modeling Methodology
- 7 Evaluation Results
- 8 Automation and MLOps
- 9 Recursive Inference and 72-Hour Forecast
- 10 API, Dashboard, Explainability and Alerts
- 11 Testing and Quality Assurance
- 12 Limitations and Production Considerations
- 13 Conclusion
- Appendix A** Repository Map
- Appendix B** Reproducibility Checklist

Verified project snapshot: the current public repository defines 35 forecasting features, a one-hour-ahead target transformation, an hourly feature workflow, recursive forecasting up to three days, API/dashboard source code, SHAP support for supported model types, and hazard classification.

1. Executive Summary

Pearls AQI Predictor is a machine-learning forecasting application for Karachi air quality. The public repository separates data acquisition, feature engineering, model training, recursive inference, API serving, dashboard presentation, explainability, hazard logic, testing and scheduled automation. The configured forecast horizon is three days, while the model target is explicitly constructed as the AQI one hour into the future.

- **Forecasting contract:** features available at hour t are used to predict AQI at hour $t+1$; the final training row is dropped because its future target is unavailable.
- **Models compared:** Naive Persistence, Random Forest, Ridge Regression and LSTM. The naive model is treated as the baseline rather than as the final ML model.
- **Recorded redesigned experiment:** Ridge produced RMSE 0.45, MAE 0.37 and R^2 0.9984 on the held-out one-hour-ahead segment; the Naive Persistence baseline produced RMSE 0.51.
- **Application layer:** Flask endpoints and a Streamlit dashboard are represented in the repository architecture and README; public hosting was not independently verified.

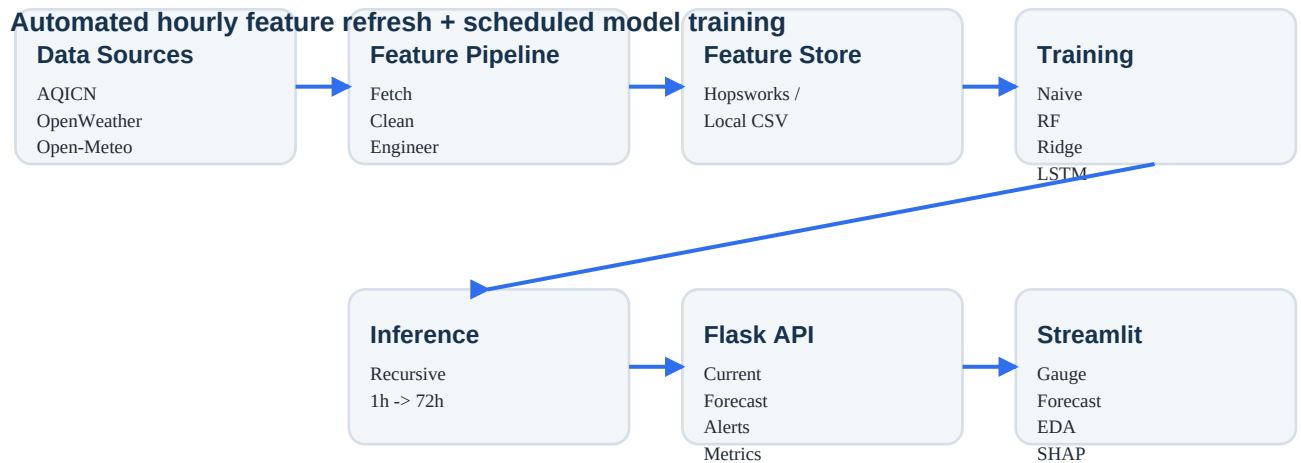
Key interpretation: the reported R^2 is a property of a particular held-out dataset and split. It does not mean the system is "99.84% accurate" in general, and it does not establish real-world performance across seasons or future environments.

2. Project Objectives and Internship Deliverables

Deliverable / expectation	Evidence in project	Assessment
1. End-to-end AQI prediction system	Data ingestion, feature engineering, model training, saved artifacts, recursive inference, API layer and dashboard source are present.	Implemented
2. Scalable / automated pipeline	Feature workflow has hourly cron scheduling and manual realtime/backfill modes; training is part of the documented automation architecture.	Implemented
3. Interactive dashboard for current and forecast AQI	README describes AQI gauge, three-day forecast, EDA/trends, SHAP explanations and hazard alerts. Source code is present; public live deployment was not independently verified.	Implemented in repository; deployment not verified
4. Detailed report	This document records architecture, data, methodology, results, automation, tests, limitations and reproducibility notes.	Provided
Multiple forecasting approaches	Naive baseline, Random Forest, Ridge and LSTM are implemented. A classical ARIMA/SARIMA model was not verified.	Implemented models; statistical time-series model not verified
Explainability and hazardous-AQI alerts	SHAP code supports supported model classes; hazard logic is present in the feature pipeline.	Implemented in repository

The assessments above describe evidence visible in the repository. They do not imply that every component is continuously deployed in production.

3. System Architecture



The application follows a modular pipeline. External data are collected, transformed into forecasting features, stored through Hopsworks or a local CSV fallback, used for model training, and then served through recursive inference. The Flask API exposes application data/services, while Streamlit provides the dashboard interface. GitHub Actions supplies scheduled orchestration.

- **Feature refresh:** the public workflow runs on an hourly cron and accepts manual realtime/backfill dispatch.
- **Training:** the training module reads the processed feature data, creates a one-hour-ahead target and evaluates models chronologically.
- **Inference:** the application can generate a recursive hourly sequence through the configured three-day horizon.

4. Data Sources and Preparation

The repository contains separate data paths for current/live information and historical training. Current AQI and pollutants are sourced through AQICN, while weather inputs are collected from OpenWeather. Historical backfill uses Open-Meteo hourly air-quality and weather data. The settings file also defines the application defaults for Karachi and the three-day forecast horizon.

Source	Role in system	Examples
AQICN	Current AQI / pollutant ingestion	AQI, PM2.5, PM10, NO2, O3, CO
OpenWeather	Current + short-range weather	Temperature, humidity, wind speed, pressure
Open-Meteo	Historical training/backfill	US AQI and historical pollutant/weather variables

A recorded 90-day backfill produced 2,160 hourly raw rows and 2,136 engineered rows; the training run then used 2,135 rows after the one-hour-ahead target transformation removes the terminal row. These counts are validation-record values from the project run history.

Data consistency limitation: historical model training uses Open-Meteo US AQI, while live AQI ingestion is represented by AQICN. This is acceptable for a prototype, but a production release should standardize the target source and definition so training and deployment measure the same quantity.

5. Feature Engineering and Leakage Control

The current public configuration defines **35 forecasting features**: four weather variables, four calendar variables, seven AQI-history/aggregation variables, and twenty pollutant lag variables. The target remains AQI.

Group	Feature content
Weather (4)	temperature, humidity, wind_speed, pressure
Calendar (4)	hour, day_of_week, month, is_weekend
AQI history (7)	lags 1/3/6/24, rolling mean 3/6, change rate
Pollutant history (20)	PM2.5, PM10, NO2, O3 and CO at lags 1/3/6/24 each

- **Target shift:** training creates `target_aqi` = AQI shifted by -1 row, making the task one-hour-ahead.
- **Rolling features:** the current public feature code shifts AQI before computing the rolling mean, so the feature excludes the row target.
- **Change rate:** the current implementation calculates percentage change from prior AQI values rather than using the target row itself.
- **Future feature row:** inference uses future weather when available plus calendar features and historical AQI/pollutant information; it does not require the future AQI value.

6. Forecasting and Modeling Methodology

Model evaluation uses chronological time-series logic rather than a random shuffle. The training implementation constructs a one-hour-ahead target and evaluates a final chronological test segment after TimeSeriesSplit is applied. The goal is to compare increasingly complex models against a persistence baseline.

Model	Type / role	Implementation summary
Naive Persistence	Baseline	$AQI(t+1) = AQI(t)$
Random Forest	Non-linear tabular ML	300 trees; max depth 12; min split 4; random_state 42.
Ridge Regression	Linear ML	StandardScaler preprocessing; alpha = 1.0.
LSTM	Sequence deep learning	24-hour sequence context in inference; used as the deep-learning comparator.

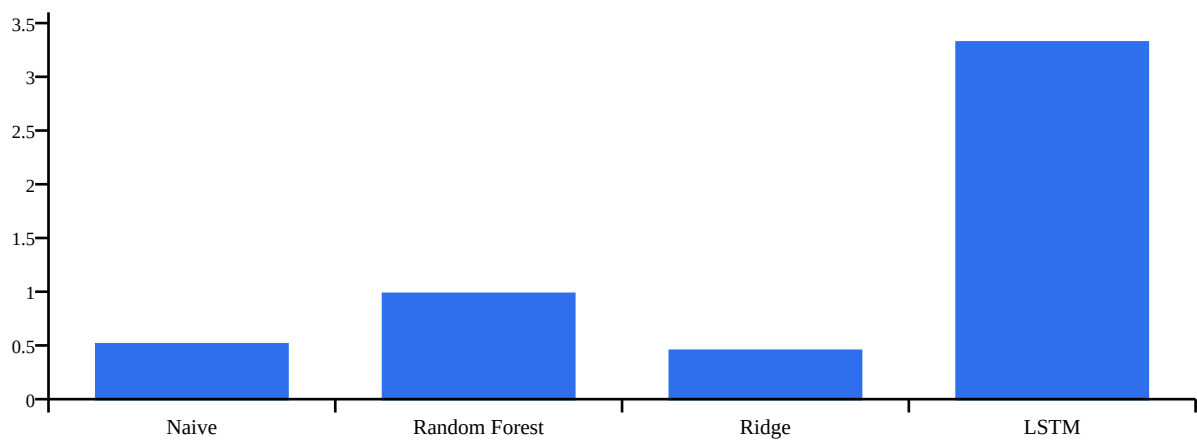
Model-selection rule: the recorded workflow chooses the best machine-learning model by RMSE and separately checks whether that model beats the naive baseline. In the recorded redesigned experiment, Ridge did beat the naive RMSE.

7. Evaluation Results

The metrics below are the recorded results from the redesigned one-hour-ahead experiment. They should be read as held-out regression metrics for that experiment, not as universal real-world accuracy claims.

Model	RMSE	MAE	R ²	Result
Naive Persistence	0.51	0.25	0.9978	Baseline
Random Forest	0.98	0.75	0.9921	Worse than baseline RMSE
Ridge	0.45	0.37	0.9984	Best ML model
LSTM	3.32	2.95	0.9151	Highest error in recorded run

Recorded held-out RMSE (one-hour-ahead experiment)



Ridge reduced RMSE from 0.51 (naive) to 0.45, an improvement of approximately 11.8% relative to the baseline RMSE. This is a useful experimental result because the persistence baseline is strong for highly autocorrelated AQI series; however, it does not validate the same error rate for every future horizon or operating condition.

8. Automation and MLOps

GitHub Actions is the main orchestration mechanism. The feature workflow is explicitly scheduled hourly and supports both realtime and backfill execution. It runs Python 3.11, installs requirements, executes the feature pipeline and, on manual backfill, can commit the refreshed processed dataset.

Workflow / component	Trigger	Main responsibility
Feature Pipeline	Hourly cron + manual dispatch	Fetch data, engineer features, write/update processed data
Training Pipeline	Scheduled / manual workflow documented by project	Train and compare models, save artifacts and metrics
Backfill mode	Manual input	Historical feature generation for a requested number of days
Artifact set	Produced by training	Models, scalers, feature-column definition, best-model reference, metrics

- **Secrets:** workflow configuration passes API credentials via GitHub Secrets rather than embedding them in the workflow.
- **Feature store:** README documents Hopsworks/local CSV as alternatives.
- **Operational caveat:** project validation previously recorded a local fallback when a Hopsworks-related dependency was unavailable. This means the system should not be described as Hopsworks-only in the validated environment.

9. Recursive Inference and 72-Hour Forecast

The inference module loads the selected model and preprocessing artifacts, reads recent history, creates a future feature row, predicts the next AQI, appends that prediction to AQI history, and repeats. With the default three-day configuration, the application produces up to 72 hourly predictions.

Step	Operation
1	Load the best model, scaler(s) and saved feature-column list.
2	Read recent historical feature context and initialize AQI/pollutant history.
3	Construct the next-hour feature vector from future weather when available, calendar variables and historical AQI/pollutant information.
4	Generate one-hour-ahead AQI prediction.
5	Append the prediction to AQI history so subsequent steps can be generated recursively.
6	Aggregate hourly forecasts into daily average, minimum, maximum and AQI category summaries.

Long-horizon warning: recursive forecasting compounds error because earlier predictions become inputs to later predictions. The strongest recorded evaluation is one-hour-ahead; a dedicated 1-to-72-hour error curve was not verified.

10. API, Dashboard, Explainability and Alerts

The README describes a Flask API and Streamlit dashboard. The documented API includes health, current AQI, forecast, daily forecast, alerts, history and model metrics. The dashboard is described with an AQI gauge, three-day forecast, EDA/trend views, SHAP explanations and hazard alerts.

Endpoint	Purpose
/health	Service health check
/api/current	Latest AQI
/api/forecast	Hourly forecast
/api/forecast/daily	Daily forecast summary
/api/alerts	AQI hazard/alert information
/api/history?hours=48	Historical readings
/api/metrics	Model metrics

- **SHAP:** the repository provides explainability for supported tree and linear model families; SHAP is not treated here as validated for every model type.
- **Hazard logic:** the feature pipeline includes fuzzy/threshold hazard classification returning a dominant risk class.
- **Dashboard status:** source code and documented views are present, but this review did not independently confirm a continuously reachable public deployment URL.

11. Testing and Quality Assurance

The repository contains automated tests covering forecasting behavior, feature engineering and hazard functionality. During the final development validation, the successful training run reported 11 tests passed before model completion. This evidence supports the core logic checks, but it is not equivalent to full end-to-end production testing against all external services.

Area	What is checked
Feature engineering	Time features, lag features, rolling means, change-rate behavior and missing-value handling
Forecasting contract	One-hour target shift and prevention of direct future-target inputs
Recursive inference	Predictions are appended to history for later forecast steps
Hazard logic	Risk category / membership behavior
Pipeline execution	GitHub Actions feature and training workflows were exercised during development

Quality boundary: automated unit/logic checks and successful workflow executions demonstrate engineering correctness for the tested paths. They do not, by themselves, prove live API reliability, cloud-service uptime, or long-term model accuracy.

12. Limitations and Production Considerations

Topic	Accurate current-state statement
Target-source consistency	Historical training uses Open-Meteo US AQI while live AQI ingestion is associated with AQICN. Standardize the target source/definition before claiming production accuracy.
Long-horizon validation	72-hour recursive generation is implemented; the strongest recorded quantitative benchmark is one-hour-ahead.
Public deployment	Repository source for Flask/Streamlit is present; a public endpoint was not independently verified during this report review.
Classical statistical model	ARIMA/SARIMA was not verified in the final implementation. Do not claim it was completed.
Feature-store dependency	Hopsworks support is represented, but local fallback was observed during validation.
Dataset breadth	The recorded historical backfill covers 90 days. Multi-season or multi-year evaluation would provide stronger evidence of generalization.
Production reproducibility	Keep the 35-feature configuration, trained artifacts and inference code synchronized in the same release.

Highest-value next improvements: standardize the AQI target source, validate the deployed API/dashboard, measure error for each forecast horizon, expand the historical dataset across seasons, and add ARIMA/SARIMA if the internship rubric explicitly requires classical statistical forecasting.

13. Conclusion

Pearls AQI Predictor demonstrates an end-to-end applied machine-learning workflow for AQI forecasting. The current public repository provides 35 configured forecasting features, one-hour-ahead target construction, multiple model families, recursive inference, scheduled feature automation, model artifacts, an API layer, dashboard code, explainability and hazard logic.

The recorded redesigned experiment selected Ridge Regression as the best machine-learning model with RMSE 0.45, MAE 0.37 and R^2 0.9984, while the persistence baseline recorded RMSE 0.51. The correct technical interpretation is that Ridge performed better on that held-out experiment; it is not a claim that the application is 99.84% accurate in all settings.

For internship assessment, the most defensible summary is: **an end-to-end, automated AQI forecasting prototype with a working model-development and application architecture, accompanied by explicit limitations around source consistency, long-horizon validation and deployment verification.**

Final report integrity: no unsupported ARIMA/SARIMA implementation, no unverified public deployment, and no misleading percentage-accuracy claim is presented as a completed fact.

Appendix A. Repository Map

Path	Purpose
config/settings.py	Central settings, city, forecast horizon, alerts and 35 FEATURE_COLUMNS.
src/feature_pipeline/	Data fetching, feature engineering, feature-store access, pipeline runner and hazard logic.
src/training_pipeline/	Model training, model components and explainability.
src/inference_pipeline/predict.py	Recursive one-hour-ahead inference and up to 72-hour forecast.
src/api/app.py	Flask REST API.
src/dashboard/app.py	Streamlit dashboard.
tests/	Automated tests for forecasting, feature and hazard behavior.
models/	Saved model/preprocessing artifacts and evaluation outputs.
.github/workflows/	Scheduled GitHub Actions automation.
data/processed/	Processed feature data and generated plots / outputs.

The repository README also documents quick-start commands for backfill, training, explainability, EDA, Flask and Streamlit execution.

Appendix B. Reproducibility Checklist

Check	Expected action / result
Environment	Create the environment from requirements.txt and configure required API keys.
Historical data	Run the documented 90-day backfill command when historical data are needed.
Training	Run the training module; verify feature count, chronological split and saved artifacts.
Explainability	Run the SHAP module for supported model families and inspect generated plots.
EDA	Run the repository EDA script and inspect outputs before interpreting trends.
API	Start Flask and verify health/current/forecast/alert/metric routes locally.
Dashboard	Start Streamlit and verify current AQI, forecast, EDA, SHAP and alert views locally.
CI/CD	Check GitHub Actions history for feature/training workflow success and inspect warnings separately from failures.
Release integrity	Keep config/settings.py, feature engineering, trained feature_cols, model artifacts and inference code synchronized before final deployment.

Repository: github.com/hassantheshah/pearls-aqi-predictor

Report date: 06 September 2026